

RESEARCH VISIT REPORT

Spoke Student Visit to IIT Hyderabad — ANRF PAIR Programme

Student	Paidipelli Bheemraj
Department	Department of Computer Science and Artificial Intelligence
Home Institution	Central University of Andhra Pradesh
Email	paidipellibheemraj1444@gmail.com
Visit Period	17 May 2026 – 17 July 2026
Focus Area	Embedded Systems, Digital Electronics and Microcontroller-Based Design
Project Track	Standalone Embedded Systems on the ATmega328P — Digital Clock and Scientific Calculator
Faculty Mentor	Prof. G. V. V. Sharma
Hosting Department	Department of Electrical Engineering, IIT Hyderabad

Overview

I am Paidipelli Bheemraj, a postgraduate student in the Department of Computer Science and Artificial Intelligence at the Central University of Andhra Pradesh. Between 17 May 2026 and 10 July 2026, I completed a research attachment at the Department of Electrical Engineering, IIT Hyderabad, hosted by Prof. G. V. V. Sharma under the ANRF PAIR Research Programme. The attachment was built around embedded systems and digital electronics, and it gave me a chance to move a set of concepts I had only seen on paper into working hardware.

Rather than a single project, the programme was structured as a progression: it started with the basics of combinational and sequential logic, moved into low-level assembly programming and Linux-based development tooling, and culminated in two standalone embedded builds — a digital clock and a scientific calculator — built around the ATmega328P microcontroller.

Goals of the Attachment

The attachment was designed to close the gap between the theory of digital systems and the practice of building them. Concretely, that meant getting comfortable with the Arduino Uno and the ATmega328P outside of it, learning to read and debug circuits on a breadboard and later on a soldered board, and building the habit of documenting engineering work properly using LaTeX rather than ad hoc notes.

Toolchain and Platforms

Over the course of the programme I worked hands-on with the following hardware and software:

- Arduino Uno development board and the standalone ATmega328P-PU microcontroller
- Seven-segment displays and 16×2 character LCD modules
- 7447 BCD-to-seven-segment decoder IC and the 7474 dual D flip-flop IC
- 5×5 matrix keypad for user input
- Termux, ArduinoDroid, Neovim and Ranger for a terminal-first Linux development workflow

- LaTeX / Overleaf for report and documentation preparation
- CAD software for enclosure modelling

Work Carried Out

1. Getting the Environment Running

The first stretch of the programme was spent setting up a working development environment rather than writing code. I installed and configured ArduinoDroid for on-device Arduino development, set up Termux with the packages needed for an embedded toolchain, and got comfortable with the Arduino IDE workflow — write, compile, upload, debug. Alongside this I was introduced to breadboarding, basic component identification, and the 7447 decoder as a first practical example of interfacing a digital IC with the Arduino.

2. Combinational Logic: Seven-Segment Displays

With the environment in place, I worked through seven-segment display interfacing in more depth. This included:

- Understanding common-anode seven-segment display behaviour and wiring it to the Arduino
- Verifying the 7447 decoder's truth table directly against hardware rather than the datasheet alone
- Writing an incrementing-decoder program and confirming correct digit output
- Working through Karnaugh Map simplification problems by hand and checking the results against the implemented logic
- Debugging display glitches that came from incorrect segment mapping

3. Sequential Logic and Assembly Programming

The next phase moved from combinational to sequential circuits. I studied the 7474 dual D flip-flop, used it to build a decade counter, and was introduced to finite state machines as a way of thinking about circuit behaviour over time. In parallel, I set up an AVR Assembly Language Programming environment inside Termux and worked through several assembly programs to see, at a lower level than Arduino's C++ wrapper, how instructions actually drive the microcontroller.

- Verified 7474-based decade counter behaviour on hardware
- Studied finite state machine design as a modelling tool
- Wrote, assembled and ran AVR ALP programs via Termux
- Cross-checked digital logic and Karnaugh Map exercises against sequential circuit outputs

4. Linux Tooling and Technical Documentation

A parallel thread throughout the programme was becoming productive in a Linux environment. I set up a dual-boot Linux installation, adopted Neovim as my primary editor, and used Ranger for terminal-based file navigation. On the documentation side, I learned LaTeX and Overleaf well enough to typeset digital-logic and GATE-style problem solutions, compiled documents directly from Neovim, and pushed source code and reports to GitHub for version control.

5. Project — Boolean Logic Circuit Analysis (GATE-Style Problem)

Alongside the two hardware builds, I independently worked through a GATE-style digital logic problem: given a three-input combinational circuit (inputs U , V , W) built from three OR gates feeding into an AND stage, derive its Boolean expression, simplify it, and verify the simplification experimentally. This was solved on my own end to end — from reading the circuit off the diagram to getting the simplified logic running correctly on hardware. The three OR outputs $X1 = U + V$, $X2 = U + W$ and $X3 = V + W$ AND together to give $I = (U + V)(U + W)(V + W)$, which Boolean algebra reduces to $I = U(V + W)$ — verified both against an 8-row truth table and by running the original and simplified expressions side by side on an Arduino UNO.

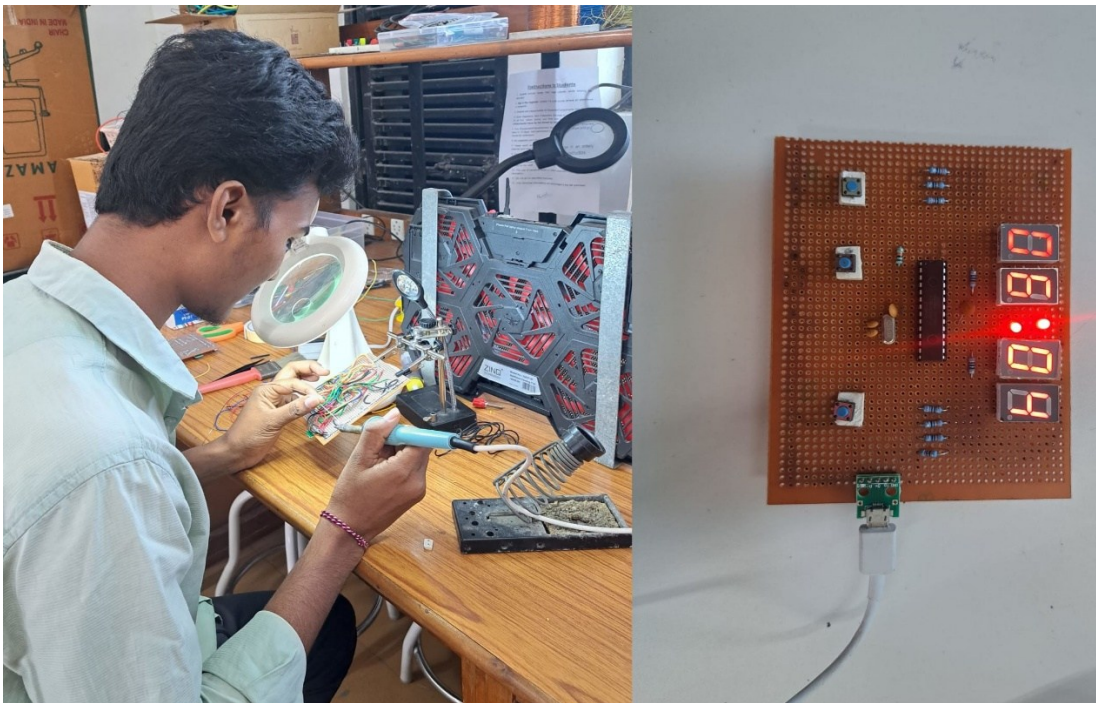
On the hardware side, three push buttons (INPUT_PULLUP, active-low) on pins D2–D4 supplied U, V and W, with an LED through a 220 Ω resistor on D13 as the output, all logged live over Serial. I also wrote an equivalent AVR Assembly version of the same logic, which matched the C++ implementation on every input combination.

- Derived and simplified the Boolean expression: $I = (U + V)(U + W)(V + W) \rightarrow I = U(V + W)$
- Verified the simplification against a hand-built truth table and live hardware
- Implemented and cross-checked the logic in both Arduino C++ and AVR Assembly
- Compared gate counts before/after simplification — three logic levels reduced to two

6. Project — Standalone Digital Clock

The first of the two capstone builds was a standalone digital clock, taken from an Arduino Uno prototype through to a self-contained ATmega328P-PU circuit. Key milestones included:

- Prototyping the clock logic and display on the Arduino Uno
- Burning the bootloader onto a bare ATmega328P using an Arduino-as-ISP setup
- Designing and validating the crystal oscillator and reset circuitry needed for standalone operation
- Iterating on the firmware to add features and fix bugs as they surfaced
- Solving the mechanical switch-debounce problem on the push buttons
- Soldering the full circuit onto dot board and validating it post-assembly
- Modelling an enclosure in CAD and testing snap-fit, bump-and-hole and screw-based closure options
- Writing up the project formally in LaTeX after multiple build-and-fix iterations

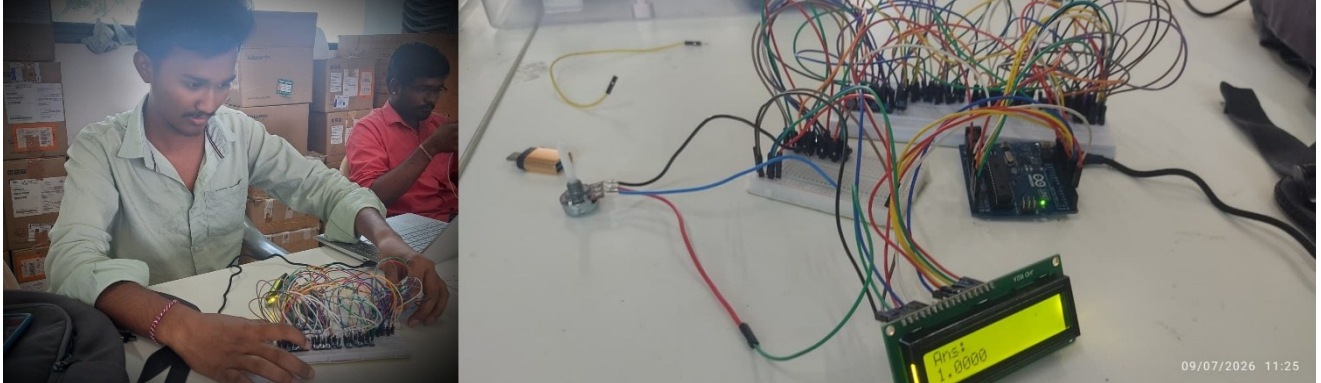


7. Project — Scientific Calculator

The second build was a scientific calculator on the Arduino Uno platform, combining a 16×2 LCD and a 5×5 matrix keypad to support both ordinary arithmetic and a set of scientific functions. Since every physical key on the 5×5 keypad was already assigned, extending the calculator with inverse trigonometric functions and other extras had to be done in software: a mode flag that reinterprets a handful of existing keys, toggled through a repurposed key with a two-step press sequence so it isn't triggered by accident mid-calculation. Domain validation was hardcoded for every function with a restricted input range — arcsin/arccos outside $[-1, 1]$, square root of a negative number, log of a non-

positive number, and division by zero — so the firmware fails gracefully instead of producing garbage. Work here included:

- Building the base hardware circuit and wiring the LCD and keypad
- Developing and iterating on the calculator firmware
- Improving the accuracy of the trigonometric function implementations
- Implementing and validating numerical routines using the fourth-order Runge–Kutta (RK4) method
- Running circuit-level verification alongside software tuning for performance
- Fixing firmware bugs uncovered during end-to-end testing
- A brief introduction to FPGAs and where they fit relative to microcontroller-based design
- Documenting the finished calculator project in LaTeX



Skills Developed

The table below summarises the main technical areas the attachment touched:

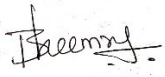
Digital Logic	Combinational and sequential circuit design, 7447 decoder, 7474 flip-flop, decade counters, Karnaugh Maps, finite state machines
Embedded Firmware	Arduino C++, modular firmware design, hardcoded domain validation, mode-flag driven UI logic, numerical methods (RK4)
Low-Level Programming	AVR Assembly Language Programming, understanding the Arduino C++ abstraction from the instruction level up
Hardware	Breadboarding, soldering onto dot board, bootloader burning via Arduino-as-ISP, oscillator and reset circuit design, debounce handling
Tooling	Linux (dual-boot), Termux, Neovim, Ranger, GitHub-based version control
Documentation	LaTeX and Overleaf for structured technical write-ups, CAD for enclosure modelling

What I Take Away From This

Beyond the finished projects, the attachment left me with a much clearer picture of the full embedded development cycle — circuit design, firmware, soldering, enclosure design, debugging and documentation — as one connected process rather than separate skills. Working through these problems on my own gave me a good sense of where things typically break (debounce, oscillator stability, decoder wiring) and how to isolate a fault methodically instead of guessing. The habit of writing things up in LaTeX as I went, rather than after the fact, is one I plan to keep — it made documentation far less painful and forced me to actually understand a circuit well enough to explain it.

Acknowledgement

I am grateful to Prof. G. V. V. Sharma and the Department of Electrical Engineering, IIT Hyderabad, for hosting me under the ANRF PAIR Research Programme and for the guidance and lab access that made this hands-on work possible. Thanks are also due to the research scholars and lab members who were generous with their time whenever I got stuck. This attachment has sharpened my practical footing in embedded systems and digital electronics far more than I expected going in, and I expect it to shape how I approach hardware-adjacent work going forward.



Student Signature (Visiting)

Date: 17-07-2026



Host Faculty Signature (IITH)

Date: 17-07-2026